C++: Operatoren

Carsten Gips (HSBI)

TL;DR
In C++ können existierende Operatoren überladen werden, etwa für die Nutzung mit eigenen Klassen. Dabei kann die Überladung innerhalb einer Klassendefinition passieren (analog zur Implementierung einer Methode) oder außerhalb der Klasse (analog zur Definition einer überladenen Funktion).
Beim Überladen in einer Klasse hat der Operator nur einen Parameter (beim Aufruf das Objekt auf der rechten Seite) und man kann auf die Attribute der Klasse direkt zugreifen. Bei der Überladung außerhalb der Klasse hat der Operator zwei Parameter und darf nicht auf die Attribute der Klasse zugreifen.
Man kann Funktionen, Methoden/Operatoren und Klassen als `friend` einer Klasse deklarieren. Damit bricht man die Kapselung auf und erlaubt den Freunden den direkten Zugriff auf die internen Attribute einer Klasse.
Um bei der Implementierung von Post- und Präfix-Operatoren die Variante für den Compiler unterscheidbar zu machen, hat die Signatur der Postfix-Variante einen Dummy-Parameter vom Typ `int`. Dieser wird beim Aufruf aber nicht genutzt.
Videos

- VL C++: Operatoren

---

1 Überladen von Operatoren *in* Klassen

```
MyString a, b("hallo");
a = b;      // ???
```

```
a.operator=(b);
```

Aufruf `a=b` ist äquivalent zu `a.operator=(b)`

Überladen ähnlich wie bei Methoden:

```
class MyString {
    MyString &operator=(const MyString &s) {
        if (this != &s) {
            // mach was :-)
        }
        return *this;
    }
};
```

Analog weitere Operatoren, etwa `operator==`, `operator+`, … überladen

2 Überladen von Operatoren *außerhalb* von Klassen

```
MyString a("hallo");
cout << a << endl;
```

```
class MyString {
    ostream &operator<<(ostream &o) { return o << str; }
};
```

So funktioniert das leider nicht!

- Erinnerung: `cout << a` entspricht `cout.operator<<(a)`
  - Operator kann nicht in `MyString` überladen werden!
  - Klasse `ostream` müsste erweitert werden
    => Geht aber nicht, da System-weite Klasse!

=> Lösung: Operator **außerhalb** der Klasse überladen => 2 Parameter

3 Überladen von Operatoren *außerhalb* von Klassen (cnt.)

Operator außerhalb der Klasse überladen => 2 Parameter

```
ostream &operator<<(ostream &out, const MyString &s) {
    return out << s.str;
}
```

- Nachteil: Benötigt Zugriff auf Klassen-Interna
  - entweder umständlich über Getter-Funktionen
  - oder als `friend` der Klasse `MyString` deklarieren
    Alternativ Zugriffsmethoden (aka *Getter*) nutzen wie `toString()` …

Anmerkung: Rückgabe der Referenz auf den Stream erlaubt die typische Verkettung: `cout << s1 << s2 << endl;`

4 Meine Freunde dürfen in mein Wohnzimmer

```
void test();

class TestDummy {
    int ganzTolleMethode();
};

```

```

class Dummy {
    private:
        int *value;

    friend class TestDummy;
    friend int TestDummy::ganzTolleMethode();
    friend void test();
};
```

5 (Fast) alle Operatoren lassen sich überladen

- Alle normalen arithmetischen Operatoren
- Zuweisung, Vergleich, Ein-/Ausgabe
- Index-Operator `[]`, Pointer-Dereferenzierung `*` und `->`, sowie `()`, **`new`** und `delete` (auch in `[]`-Form)

- **Ausnahmen**:
  1. **.**
  2. **::**
  3. **?:**
  4. **sizeof**

- Anmerkungen:
  - Beim Überladen muss die Arität erhalten bleiben
  - Nur *existierende* Operatoren lassen sich überladen
    => Es lassen sich keine neuen Operatoren erschaffen

Vgl. Tabelle 9.1 (S. 318) im Breymann (2011)

6 Implizite Typkonvertierungen bei Aufruf

```
MyString s;
s != "123";      // ???
"123" != s;      // ???
```

- Operatoren **in** Klasse überladen: Typ der linken Seite muss **exakt** passen

```
class MyString {
public:
    MyString(const char *s = "");
    bool operator!=(const MyString&);
};
```

```
6
7 MyString s;
8 s != "123";    // impliziter Aufruf des Konstruktors, danach MyString::
    operator!=
9 "123" != s;    // KEIN operator!=(char*, MyString&) vorhanden!
```

Das ist letztlich wie bei einem Methodenaufruf: Um die richtige Methode aufzurufen, muss der Typ (die Klasse) des Objekts bekannt sein.

- Operatoren **außerhalb** überladen: Konvertierung auf *beiden* Seiten möglich

```
1 class MyString {
2 public:
3     MyString(const char *s = "");
4 };
5 bool operator!=(const MyString&, const MyString&);
```

NIEMALS beide Formen gleichzeitig für einen Operator implementieren!

7 Anmerkung zu "++" und "- -" Operatoren: Präfix und Postfix

- Präfix: `o1 = ++o2;`
  - Objekt soll **vor Auswertung** inkrementiert werden
  - Signatur: `Typ &operator++()`

- Postfix: `o1 = o2++;`
  - Objekt soll erst **nach Auswertung** inkrementiert werden
  - Signatur: `Typ operator++(int)` (=> `int` dient nur zur Unterscheidung der Präfix-Variante, wird **nie** benutzt)

8 Weitere Anmerkungen

- Operatoren werden **nicht** vom System zusammengesetzt
  - `operator+` und `operator+=` sind zwei verschiedene Operatoren!
  - Implementierung ist prinzipiell unabhängig!
    => Erwartung: `operator+= == (operator+ + operator=)`

- Operatoren lassen sich in C++ verketten:

```
1 Dummy a(0); Dummy b(1); Dummy c(2);
2 a = b = c;  // a.operator=(b.operator=(c));
```

- Übertreiben Sie nicht!

```
Firma f;
Person p;
f += p;  // ??!
```

Nutzen Sie im Zweifel lieber Methoden mit aussagekräftigen Namen!

9 Wrap-Up

- Überladen von Operatoren (innerhalb und außerhalb einer Klasse)
  - Innerhalb: 1 Parameter (Objekt auf der rechten Seite)
  - Außerhalb: 2 Parameter
- Zugriff auf Attribute: `friend` einer Klasse
- Implementierung von Post- und Präfix-Operatoren

10 Zum Nachlesen

- Breymann (2011)
- „C and C++ Reference“ (o. J.)
- Allain und Hoffer (o. J.)

Lernziele

- k2: Implizite Typkonvertierungen bei Operatoren
- k3: Überladen von Operatoren (innerhalb bzw. außerhalb einer Klasse)
- k3: Anwendung der Deklaration als friend
- k3: Implementierung von Post- und Präfix-Operatoren

Challenges

Operator "++"

Betrachten Sie die folgende Klasse:

```
class Studi {
public:
    Studi(int credits);
    ~Studi();
private:
    int *credits;
};
```

Implementieren Sie den `operator++` sowohl in der Präfix- als auch in der Postfix-Variante.

C'toren und Operatoren: Was muss noch *deklariert* werden?

```
class Studi {
public:
    Studi(int credits);
private:
    int *credits;
};

int main() {
    Studi a(1), b, *c = new Studi(99);
    b = *c+a+1;
    std::cout << "b: '" << b << "' credits" << std::endl;

    return 0;
}
```

Schreiben Sie Code, damit folgender Code kompiliert:

```
test wuppie;
bool fluppie = wuppie(3);
```

Quellen

Allain, A., und A. Hoffer. o. J. „C Programming and C++ Programming". Zugegriffen 16. September 2024. https://www.cprogramming.com/.

Breymann, U. 2011. *Der C++ Programmierer*. 2. Auflage. Hanser.

„C and C++ Reference". o. J. cppreference.com. Zugegriffen 16. September 2024. https://en.cppreference.com/w/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** a10718a (Remove Hugo: tooling: shift headings (use h1 as top-level headings), 2025-07-25)